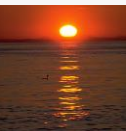




# Database Management System (DBMS)

**Curated by:**

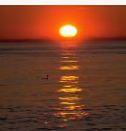
Tohida Rehman  
Assistant Professor  
Department of Information Technology  
Jadavpur University





# Closure of a Set of Functional Dependencies

- Given a set  $F$  set of functional dependencies, there are certain other functional dependencies that are logically implied by  $F$ .
  - For example: If  $A \rightarrow B$  and  $B \rightarrow C$ , then we can infer that  $A \rightarrow C$
- The set of **all** functional dependencies logically implied by  $F$  is the **closure** of  $F$ .
- We denote the **closure** of  $F$  by  $F^+$ .
- We can find all of  $F^+$  by applying Armstrong's Axioms:
  - if  $\beta \subseteq \alpha$ , then  $\alpha \rightarrow \beta$  **(reflexivity)**
  - if  $\alpha \rightarrow \beta$ , then  $\gamma \alpha \rightarrow \gamma \beta$  **(augmentation)**
  - if  $\alpha \rightarrow \beta$ , and  $\beta \rightarrow \gamma$ , then  $\alpha \rightarrow \gamma$  **(transitivity)**
- These rules are
  - **sound** (generate only functional dependencies that actually hold) and
  - **complete** (generate all functional dependencies that hold).





# Example

□  $R = (A, B, C, G, H, I)$

$F = \{ A \rightarrow B$

$A \rightarrow C$

$CG \rightarrow H$

$CG \rightarrow I$

$B \rightarrow H\}$

□ some members of  $F^+$

□  $A \rightarrow H$

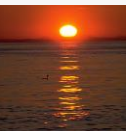
▶ by transitivity from  $A \rightarrow B$  and  $B \rightarrow H$

□  $AG \rightarrow I$

▶ by augmenting  $A \rightarrow C$  with  $G$ , to get  $AG \rightarrow CG$   
and then transitivity with  $CG \rightarrow I$

□  $CG \rightarrow HI$

▶ by augmenting  $CG \rightarrow I$  to infer  $CG \rightarrow CGI$ ,  
and augmenting of  $CG \rightarrow H$  to infer  $CGI \rightarrow HI$ ,  
and then transitivity





# Procedure for Computing $F^+$

- To compute the closure of a set of functional dependencies  $F$ :

$F^+ = F$

**repeat**

**for each** functional dependency  $f$  in  $F^+$

        apply reflexivity and augmentation rules on  $f$

        add the resulting functional dependencies to  $F^+$

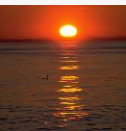
**for each** pair of functional dependencies  $f_1$  and  $f_2$  in  $F^+$

**if**  $f_1$  and  $f_2$  can be combined using transitivity

**then** add the resulting functional dependency to  $F^+$

**until**  $F^+$  does not change any further

**NOTE:** We shall see an alternative procedure for this task later





# Closure of Functional Dependencies (Cont.)

- We can further simplify manual computation of  $F^+$  by using the following additional rules.
  - If  $\alpha \rightarrow \beta$  holds and  $\alpha \rightarrow \gamma$  holds, then  $\alpha \rightarrow \beta \gamma$  holds (**union**)
  - If  $\alpha \rightarrow \beta \gamma$  holds, then  $\alpha \rightarrow \beta$  holds and  $\alpha \rightarrow \gamma$  holds (**decomposition**)
  - If  $\alpha \rightarrow \beta$  holds and  $\gamma \beta \rightarrow \delta$  holds, then  $\alpha \gamma \rightarrow \delta$  holds (**pseudotransitivity**)

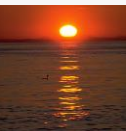
The above rules can be inferred from Armstrong's axioms.





# Example of Attribute Set Closure

- $R = (A, B, C, G, H, I)$
- $F = \{A \rightarrow B$   
 $A \rightarrow C$   
 $CG \rightarrow H$   
 $CG \rightarrow I$   
 $B \rightarrow H\}$
- $(AG)^+$ 
  1.  $result = AG$
  2.  $result = ABCG$  ( $A \rightarrow C$  and  $A \rightarrow B$ )
  3.  $result = ABCGH$  ( $CG \rightarrow H$  and  $CG \subseteq AGBC$ )
  4.  $result = ABCGHI$  ( $CG \rightarrow I$  and  $CG \subseteq AGBCH$ )
- Is  $AG$  a candidate key?
  1. Is  $AG$  a super key?
    1. Does  $AG \rightarrow R?$  == Is  $(AG)^+ \supseteq R$
  2. Is any subset of  $AG$  a superkey?
    1. Does  $A \rightarrow R?$  == Is  $(A)^+ \supseteq R$
    2. Does  $G \rightarrow R?$  == Is  $(G)^+ \supseteq R$

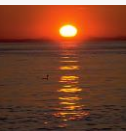




# Uses of Attribute Closure

There are several uses of the attribute closure algorithm:

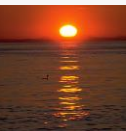
- Testing for superkey:
  - To test if  $\alpha$  is a superkey, we compute  $\alpha^+$ , and check if  $\alpha^+$  contains all attributes of  $R$ .
- Testing functional dependencies
  - To check if a functional dependency  $\alpha \rightarrow \beta$  holds (or, in other words, is in  $F^+$ ), just check if  $\beta \subseteq \alpha^+$ .
  - That is, we compute  $\alpha^+$  by using attribute closure, and then check if it contains  $\beta$ .
  - Is a simple and cheap test, and very useful
- Computing closure of  $F$ 
  - For each  $\gamma \subseteq R$ , we find the closure  $\gamma^+$ , and for each  $S \subseteq \gamma^+$ , we output a functional dependency  $\gamma \rightarrow S$ .





# Canonical Cover

- Sets of functional dependencies may have redundant dependencies that can be inferred from the others
  - For example:  $A \rightarrow C$  is redundant in:  $\{A \rightarrow B, B \rightarrow C\}$
  - Parts of a functional dependency may be redundant
    - ▶ E.g.: on RHS:  $\{A \rightarrow B, B \rightarrow C, A \rightarrow CD\}$  can be simplified to
$$\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$$
    - ▶ E.g.: on LHS:  $\{A \rightarrow B, B \rightarrow C, AC \rightarrow D\}$  can be simplified to
$$\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$$
- Intuitively, a canonical cover of  $F$  is a “minimal” set of functional dependencies equivalent to  $F$ , having no redundant dependencies or redundant parts of dependencies

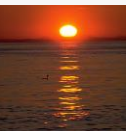






# Extraneous Attributes

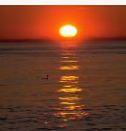
- Consider a set  $F$  of functional dependencies and the functional dependency  $\alpha \rightarrow \beta$  in  $F$ .
  - Attribute  $A$  is **extraneous** in  $\alpha$  if  $A \in \alpha$  and  $F$  logically implies  $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$ .
  - Attribute  $A$  is **extraneous** in  $\beta$  if  $A \in \beta$  and the set of functional dependencies  $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$  logically implies  $F$ .
- *Note:* implication in the opposite direction is trivial in each of the cases above, since a “stronger” functional dependency always implies a weaker one
- Example: Given  $F = \{A \rightarrow C, AB \rightarrow C\}$ 
  - $B$  is extraneous in  $AB \rightarrow C$  because  $\{A \rightarrow C, AB \rightarrow C\}$  logically implies  $A \rightarrow C$  (i.e. the result of dropping  $B$  from  $AB \rightarrow C$ ).
- Example: Given  $F = \{A \rightarrow C, AB \rightarrow CD\}$ 
  - $C$  is extraneous in  $AB \rightarrow CD$  since  $AB \rightarrow C$  can be inferred even after deleting  $C$





# Testing if an Attribute is Extraneous

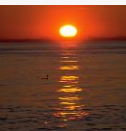
- Consider a set  $F$  of functional dependencies and the functional dependency  $\alpha \rightarrow \beta$  in  $F$ .
- To test if attribute  $A \in \alpha$  is extraneous in  $\alpha$ 
  1. compute  $(\{\alpha\} - A)^+$  using the dependencies in  $F$
  2. check that  $(\{\alpha\} - A)^+$  contains  $A$ ; if it does,  $A$  is extraneous
- To test if attribute  $A \in \beta$  is extraneous in  $\beta$ 
  1. compute  $\alpha^+$  using only the dependencies in
$$F' = (F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\},$$
  2. check that  $\alpha^+$  contains  $A$ ; if it does,  $A$  is extraneous





# Canonical Cover

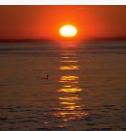
- A *canonical cover* for  $F$  is a set of dependencies  $F_c$  such that
  - $F$  logically implies all dependencies in  $F_c$ , and
  - $F_c$  logically implies all dependencies in  $F$ , and
  - No functional dependency in  $F_c$  contains an extraneous attribute, and
  - Each left side of functional dependency in  $F_c$  is unique.
- To compute a canonical cover for  $F$ :
  - repeat**
    - Use the union rule to replace any dependencies in  $F$   
 $\alpha_1 \rightarrow \beta_1$  and  $\alpha_1 \rightarrow \beta_2$  with  $\alpha_1 \rightarrow \beta_1 \beta_2$
    - Find a functional dependency  $\alpha \rightarrow \beta$  with an  
extraneous attribute either in  $\alpha$  or in  $\beta$
    - If an extraneous attribute is found, delete it from  $\alpha \rightarrow \beta$
  - until**  $F$  does not change
- Note: Union rule may become applicable after some extraneous attributes have been deleted, so it has to be re-applied





# Computing a Canonical Cover

- $R = (A, B, C)$   
 $F = \{A \rightarrow BC$   
     $B \rightarrow C$   
     $A \rightarrow B$   
     $AB \rightarrow C\}$
- Combine  $A \rightarrow BC$  and  $A \rightarrow B$  into  $A \rightarrow BC$ 
  - Set is now  $\{A \rightarrow BC, B \rightarrow C, AB \rightarrow C\}$
- $A$  is extraneous in  $AB \rightarrow C$ 
  - Check if the result of deleting  $A$  from  $AB \rightarrow C$  is implied by the other dependencies
    - ▶ Yes: in fact,  $B \rightarrow C$  is already present!
  - Set is now  $\{A \rightarrow BC, B \rightarrow C\}$
- $C$  is extraneous in  $A \rightarrow BC$ 
  - Check if  $A \rightarrow C$  is logically implied by  $A \rightarrow B$  and the other dependencies
    - ▶ Yes: using transitivity on  $A \rightarrow B$  and  $B \rightarrow C$ .
      - Can use attribute closure of  $A$  in more complex cases
- The canonical cover is:  
     $A \rightarrow B$   
     $B \rightarrow C$





# Lossless-join Decomposition

- For the case of  $R = (R_1, R_2)$ , we require that for all possible relations  $r$  on schema  $R$

$$r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$$

- A decomposition of  $R$  into  $R_1$  and  $R_2$  is lossless join if and only if at least one of the following dependencies is in  $F^+$ :
  - $R_1 \cap R_2 \rightarrow R_1$
  - $R_1 \cap R_2 \rightarrow R_2$





# Example

- $R = (A, B, C)$   
 $F = \{A \rightarrow B, B \rightarrow C\}$ 
  - Can be decomposed in two different ways

- $R_1 = (A, B), R_2 = (B, C)$ 
  - Lossless-join decomposition:

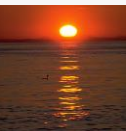
$$R_1 \cap R_2 = \{B\} \text{ and } B \rightarrow BC$$

- Dependency preserving

- $R_1 = (A, B), R_2 = (A, C)$ 
  - Lossless-join decomposition:

$$R_1 \cap R_2 = \{A\} \text{ and } A \rightarrow AB$$

- Not dependency preserving  
(cannot check  $B \rightarrow C$  without computing  $R_1 \bowtie R_2$ )





# Dependency Preservation

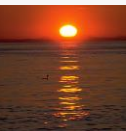
- Let  $F_i$  be the set of dependencies  $F^+$  that include only attributes in  $R_i$ .
  - ▶ A decomposition is **dependency preserving**, if
$$(F_1 \cup F_2 \cup \dots \cup F_n)^+ = F^+$$
  - ▶ If it is not, then checking updates for violation of functional dependencies may require computing joins, which is expensive.





# Testing for Dependency Preservation

- To check if a dependency  $\alpha \rightarrow \beta$  is preserved in a decomposition of  $R$  into  $R_1, R_2, \dots, R_n$  we apply the following test (with attribute closure done with respect to  $F$ )
  - $result = \alpha$   
  **while** (changes to  $result$ ) **do**  
    **for each**  $R_i$  in the decomposition  
       $t = (result \cap R_i)^+ \cap R_i$   
       $result = result \cup t$
  - If  $result$  contains all attributes in  $\beta$ , then the functional dependency  $\alpha \rightarrow \beta$  is preserved.
- We apply the test on all dependencies in  $F$  to check if a decomposition is dependency preserving
- This procedure takes polynomial time, instead of the exponential time required to compute  $F^+$  and  $(F_1 \cup F_2 \cup \dots \cup F_n)^+$







# Example

- $R = (A, B, C)$   
 $F = \{A \rightarrow B$   
 $\quad B \rightarrow C\}$   
Key =  $\{A\}$
- $R$  is not in BCNF
- Decomposition  $R_1 = (A, B), R_2 = (B, C)$ 
  - $R_1$  and  $R_2$  in BCNF
  - Lossless-join decomposition
  - Dependency preserving

